```
- 1117838570 2005.06.03 R02-M1-N0-C:J12-U11 2005-06-03-15.42.50.675872 R02-M1-N0-C:J12-U11 RAS KERNEL INFO
instruction cache parity error corrected
```

Making Sense of Logs

Same logs, three people, three different answers. And every number can be checked.

Sai Jaya Ram Challa
team ultraviolet-owls · SereneDB challenge, National AI Hackathon 2026
github.com/Jay-0903/log-intelligence-agent

Nobody reads 4.7 million log lines

```
03:24:46 KERNMC machine check interrupt
03:24:46 - instruction address: 0x00205154
03:24:47 - Machine Check Status Register
03:24:47 - machine check summary.......1
03:24:47 - data write plb error........1
03:24:47 - tlb error...................0
...
... 4,747,963 lines in BGL alone
```

BGL (a supercomputer): 4,747,963 lines
OpenStack (a cloud platform): 207,820 lines

The engineer asks

"What broke, and what happened right before?"

The team lead asks

"Is this a pattern or one bad day? Whose is it?"

The manager asks

"What should we fix first?"

Too much for any AI prompt. And a shorter answer isn't the manager's answer.

So I let the database do the reading

A question → **Claude picks a tool** → **SereneDB runs it** → **A small result** → **The answer**

from one of the three people

through MCP, a standard plug for AI tools

the database searches or counts all the data

plus the exact SQL that produced it

fact, guess and next step, kept apart

Four rules I set myself

1 **The database counts, the AI explains.**
An AI can't read 4.7 million lines, and counts must be exact.

2 **No free-form SQL.**
A made-up query can be quietly wrong, so Claude only uses tested tools.

3 **Every answer shows its SQL.**
Someone will act on these numbers.

4 **Facts and guesses stay apart.**
Logs show what happened, not why.

Each person gets their own tools

Engineer

needs the exact lines and the minutes around them

```
search_logs_fulltext
get_nearby_events
request_trace
```

Team lead

needs to tell a one-day spike from a problem that keeps coming back

```
alert_profile
alert_categories
count_alerts_by_team
trend_by_day
trend_by_week
top_nodes_for_alert
```

Manager

needs error rates and an owner for each problem

```
alert_profile
openstack_by_component
latency_stats
```

Claude reads the descriptions and picks a tool itself. I never tell it which.

16 tools in total: 13 core (with 2 shared ones for data scope and team ownership) and 3 optional ones for a 1-billion-row dataset.

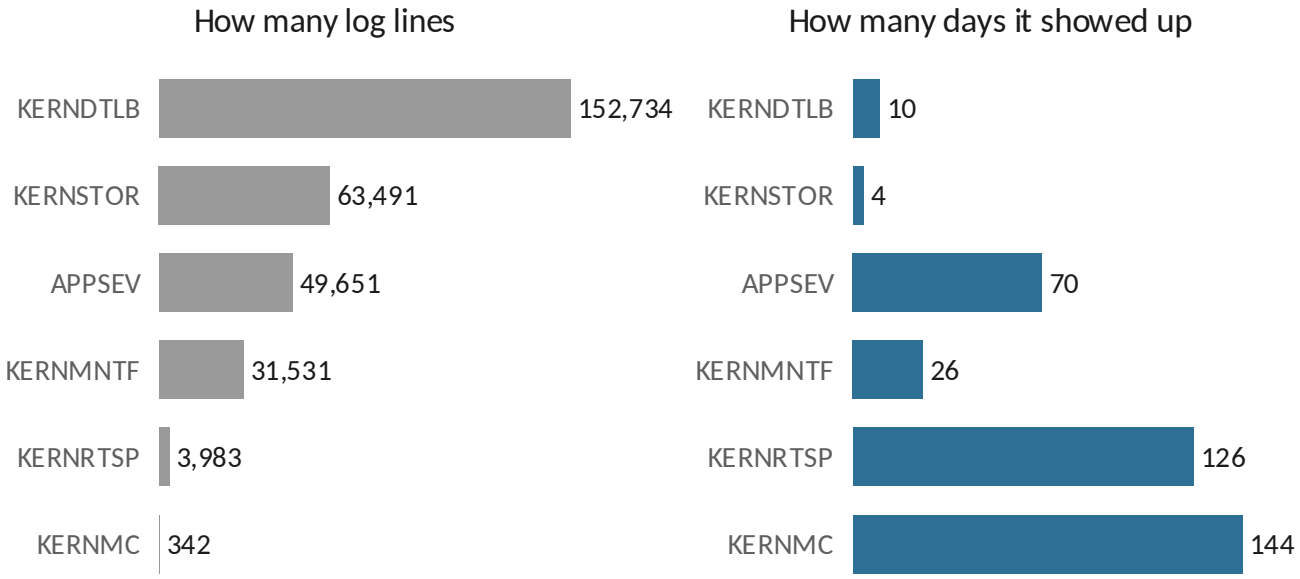
Engineer: one bad chip took down a job

I asked: "Investigate KERNRTSP alerts: which nodes and days, example lines, and what happened before and after the most recent one?"

```
3 Jan 2006 · node R27-M0-NA-C:J11-U11 · the cluster was quiet from 02:24 to 03:24
03:24:46.844 KERNMC   machine check interrupt (bit=0x1d): L2 dcache unit write data parity error
03:24:47.688 -        data write plb error.............1
                      ... about 19 seconds of state dump ...
03:25:05.504 KERNRTSP rts panic! - stopping execution
03:25:06.083 APPSEV   ciod: ... Link has been severed   (rack R25)
03:25:06.377 APPSEV   ciod: ... Link has been severed   (rack R22)
                      ... and racks R20, R24, R26, R27
```

What I can see	3,983 lines on 3,394 nodes over 126 days; this one came 19 s after a cache error
My best guess, not proven	a cache fault on that chip ended one large job
What to check next	this node, and R63-M0-NE-C:J12-U01, which has the most panics (22)

Team lead: loud isn't the same as recurring



99.6%

of KERNDTLB's 152,734 lines happened on one day, 12 June 2005. One bad day, not a pattern.

KERNMC has only 342 lines, but it showed up on 144 days.

6 of the 41 alert codes in the data · whole BGL dataset, June 2005 to January 2006 · one log line is not one incident

Manager: what I'd fix first

1	**Kernel Platform**	KERNMC (144 days) and KERNRTSP (126 days), both still active at the end *→ check the nodes that appear in both*
2	**Application Runtime**	APPSEV: 70 days on 1,024 distinct nodes *→ looks systemic, not one bad machine*
3	**API Gateway**	nova-api: 2.01% of lines are errors (2,068 of 102,868) *→ find out what that 2% is*
4	**Compute**	nova-compute: 0.20% of lines are errors *→ keep an eye on it*

One-day bursts like KERNDTLB and KERNSTOR: review them once, don't chase them.

Ranked by how long and how widely a problem shows up · owners from my ownership table · no made-up costs

Every number comes with its query

```
scope: BGL, 4,747,963 lines | BM25 full-text

SELECT id, alert, ts, node,
       event_time AS "datetime", severity,
       component, message,
       ROUND(BM25(bgl_idx.tableoid)::NUMERIC, 3)
         AS score
FROM bgl_idx
WHERE (message @@ ts_phrase('rts')
       OR message @@ ts_phrase('panic'))
  AND ts >= 1136246400 AND ts <= 1136332800
ORDER BY score DESC LIMIT 5;

-> R27-M0-NA-C:J11-U11  rts panic!  8.382
```

You can rerun every query yourself from a clean checkout of the repo.

How this saved me

Halfway through, search on the full data quietly fell back to plain text matching.

Every tool says why it falls back, so I could rule out the data and the index.

It was how the database driver sent parameters. One line fixed it.

What this can and can't tell you

It can tell you

- ✔ exact line counts over all the data
- ✔ how problems spread over machines and days
- ✔ which team owns each component
- ✔ what happened around an event

It can't tell you

- ✖ the confirmed root cause
- ✖ how many incidents (it counts lines)
- ✖ downtime, cost or customer impact
- ✖ OpenStack's labelled anomalies (I use an error-keyword rate)

Bonus: I also pointed it at 1 billion OpenTelemetry rows on S3 (1 of 50 files), without downloading anything. **Counts took 11.7 to 15.5 s.** The full-text index ran out of memory on my 6 GB laptop.

Question → evidence → recommendation

The engineer gets the exact lines and what happened around them.

The team lead sees what keeps coming back, not just what is loud.

The manager gets a short list with owners, and no made-up numbers.

Same agent, same data, and every number can be checked.

Thanks for listening! · github.com/Jay-0903/log-intelligence-agent